Film Production Database Management System

Project Report

Course	CMPE 344 – Database Systems
Submission Date	16 May 2026
GitHub Link	https://github.com/22314825/Film-Production-Database-Management-System

Name	Student No.	Role
Baran Koç	22314825	System Designer + Lead Developer
Efe Kemal Kayış	22314842	Backend Developer (Admin System, UI Implementation)
Berk Elmalı	22314621	UI Implementation + Backend Developer
Emirhan Keskin	22314856	Documentation + Debug / Bugfix + UI Enhancement

1. Introduction

The Film Production Database Management System (FPDMS) is a full-stack desktop application built with Electron (Node.js) as the application shell and a PostgreSQL database hosted on Neon (serverless Postgres) as the persistent data store. The system allows a film-production company to manage its entire catalogue of movies, cast members (actors), production staff (crew members, directors, producers), and associated financial records from a single unified interface.

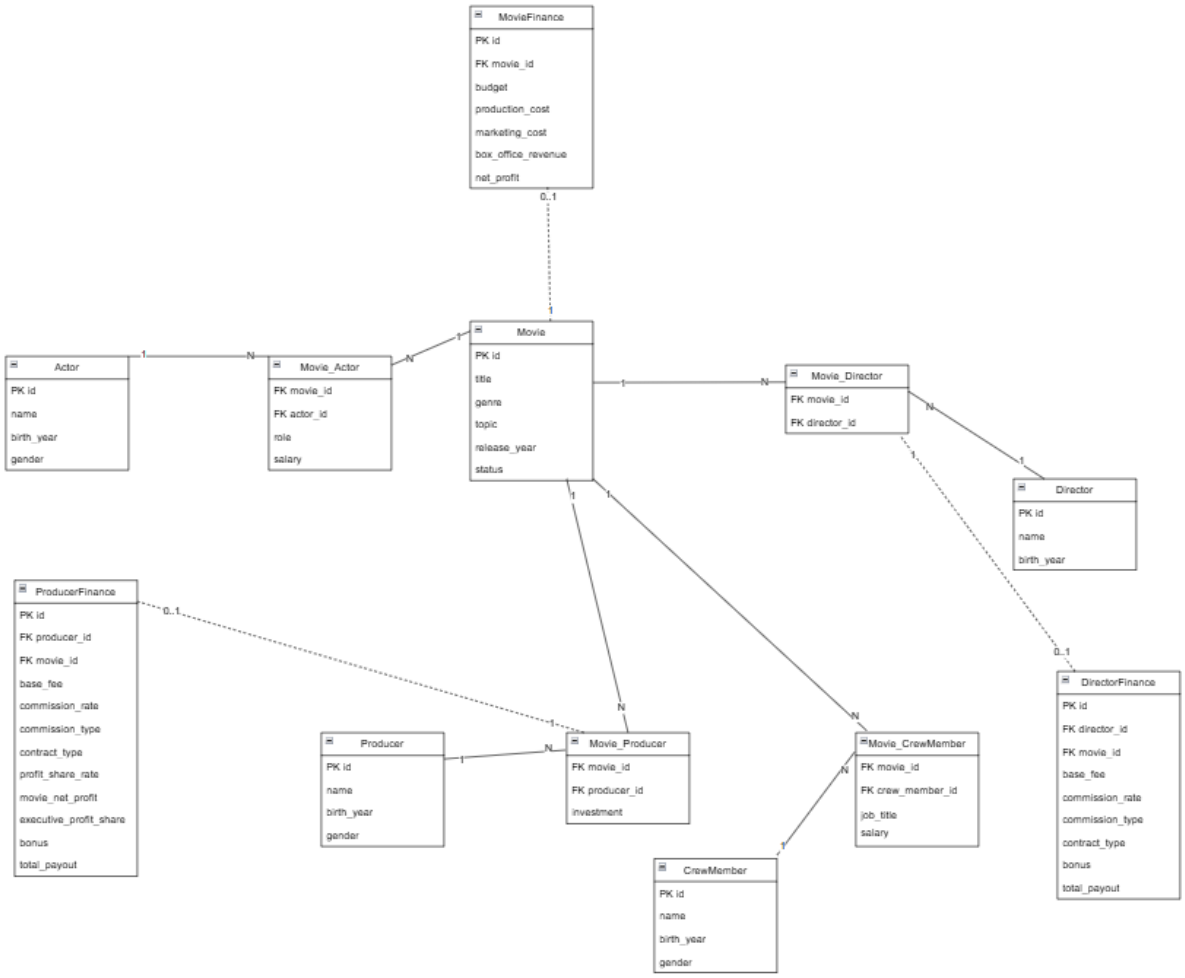
The application supports two access modes: an Admin role with full CRUD capabilities, and a read-only Spectator role. All business logic is encapsulated in PL/pgSQL stored functions and database views, keeping the Node.js layer thin and purely concerned with IPC communication between the Electron main process and the renderer.

Assumptions & Scope

- A movie progresses through two statuses: draft (in development) and published (released). A movie can only be published if it has at least one financial record.
- Actors and crew members are hired per movie; their role, job title, and salary therefore live in the junction tables (Movie_Actor, Movie_CrewMember) rather than on the person entity itself.
- Budget for a movie is derived from the sum of all producer investments and stored in MovieFinance.
- Net profit is a GENERATED ALWAYS AS column: box_office_revenue - production_cost - marketing_cost.
- Financial data for directors and producers (fees, commissions, bonuses) is tracked in dedicated DirectorFinance and ProducerFinance tables.
- The system is intended for single-user, single-company use; no multi-tenancy is implemented.
- The database is accessed through the Neon serverless driver (@neondatabase/serverless) using a connection-pool URL stored in a .env file.

2. Database

2.1 Entity-Relationship (ER) Diagram



2.2 Data Definition Language (DDL)

The schema is created through numbered migration files executed in order by migrations/migrate.js. All referential integrity is enforced via foreign-key constraints with CASCADE delete, and computed columns use the PostgreSQL GENERATED ALWAYS AS ... STORED syntax.

001_core_tables.sql — Core Entities

```
CREATE TABLE Movie (
    id           SERIAL PRIMARY KEY,
    title        VARCHAR(255) NOT NULL,
    genre        VARCHAR(100),
    topic        VARCHAR(100),
    release_year INT,
    status       VARCHAR(20) DEFAULT 'draft'
);

CREATE TABLE Producer (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(255) NOT NULL,
    birth_year INT,
    gender     VARCHAR(10)
);

CREATE TABLE Director (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(255) NOT NULL,
    birth_year INT
);

CREATE TABLE Actor (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(255) NOT NULL,
    birth_year INT,
    gender     VARCHAR(10)
);

CREATE TABLE CrewMember (
    id         SERIAL PRIMARY KEY,
    name       VARCHAR(255) NOT NULL,
    birth_year INT,
    gender     VARCHAR(10)
);
```

002_connections.sql — Junction Tables

```
-- Movie <-> Actor  (role + salary per movie)
CREATE TABLE Movie_Actor (
    movie_id  INT NOT NULL REFERENCES Movie(id)  ON DELETE CASCADE,
    actor_id  INT NOT NULL REFERENCES Actor(id)  ON DELETE CASCADE,
    role      VARCHAR(100),
    salary    DECIMAL(15, 2),
    PRIMARY KEY (movie_id, actor_id)
);

-- Movie <-> CrewMember  (job_title + salary per movie)
CREATE TABLE Movie_CrewMember (
    movie_id       INT NOT NULL REFERENCES Movie(id)       ON DELETE CASCADE,
    crew_member_id INT NOT NULL REFERENCES CrewMember(id)  ON DELETE CASCADE,
    job_title      VARCHAR(100),
    salary         DECIMAL(15, 2),
    PRIMARY KEY (movie_id, crew_member_id)
);

-- Movie <-> Director
CREATE TABLE Movie_Director (
    movie_id    INT NOT NULL REFERENCES Movie(id)    ON DELETE CASCADE,
    director_id INT NOT NULL REFERENCES Director(id) ON DELETE CASCADE,
    PRIMARY KEY (movie_id, director_id)
);

-- Movie <-> Producer  (investment per movie)
CREATE TABLE Movie_Producer (
    movie_id    INT NOT NULL REFERENCES Movie(id)    ON DELETE CASCADE,
    producer_id INT NOT NULL REFERENCES Producer(id) ON DELETE CASCADE,
    investment  DECIMAL(15, 2) NOT NULL,
    PRIMARY KEY (movie_id, producer_id)
);
```

003_finance.sql — Finance Tables & ENUM Types

```
CREATE TYPE commission_type AS ENUM ('gross', 'net', 'backend');
CREATE TYPE contract_type   AS ENUM ('flat', 'percentage', 'hybrid');

CREATE TABLE MovieFinance (
    id                SERIAL PRIMARY KEY,
    movie_id          INT          NOT NULL UNIQUE REFERENCES Movie(id) ON DELETE CASCADE,
    budget            DECIMAL(15, 2),
    production_cost   DECIMAL(15, 2),
    marketing_cost    DECIMAL(15, 2),
    box_office_revenue DECIMAL(15, 2),
    net_profit        DECIMAL(15, 2)
        GENERATED ALWAYS AS (box_office_revenue - production_cost - marketing_cost) STORED
);

CREATE TABLE DirectorFinance (
    id              SERIAL PRIMARY KEY,
    director_id     INT NOT NULL REFERENCES Director(id) ON DELETE CASCADE,
    movie_id        INT NOT NULL REFERENCES Movie(id)    ON DELETE CASCADE,
    base_fee        DECIMAL(15, 2),
    commission_rate DECIMAL(5, 2),
    commission_type commission_type,
    contract_type   contract_type,
    bonus           DECIMAL(15, 2),
    total_payout    DECIMAL(15, 2),
    UNIQUE (director_id, movie_id)
);

CREATE TABLE ProducerFinance (
    id                    SERIAL PRIMARY KEY,
    producer_id           INT NOT NULL REFERENCES Producer(id) ON DELETE CASCADE,
    movie_id              INT NOT NULL REFERENCES Movie(id)    ON DELETE CASCADE,
    base_fee              DECIMAL(15, 2),
    commission_rate       DECIMAL(5, 2),
    commission_type       commission_type,
    contract_type         contract_type,
    profit_share_rate     DECIMAL(5, 2),
    movie_net_profit      DECIMAL(15, 2),
    executive_profit_share DECIMAL(15, 2)
        GENERATED ALWAYS AS (movie_net_profit * profit_share_rate / 100.0) STORED,
    bonus                 DECIMAL(15, 2),
    total_payout          DECIMAL(15, 2),
    UNIQUE (producer_id, movie_id)
);
```

2.3 Data Manipulation Language (DML)

All inserts, updates, and deletes are performed through PL/pgSQL stored functions rather than raw SQL statements in the application layer.

INSERT — Sample seed data

```
INSERT INTO Movie (title, genre, topic, release_year, status) VALUES
    ('Midnight Siege',  'Action', 'Cyber Heist',       2025, 'published'),
    ('Glass Horizon',   'Drama',  'Family Legacy',      2024, 'published'),
    ('Orbital Drift',   'Sci-Fi', 'Deep Space Rescue',  2026, 'published');

INSERT INTO Actor (name, birth_year, gender) VALUES
    ('Sofia Reyes', 1991, 'Female'),
    ('Noah Carter',  1988, 'Male'),
    ('Liam Bennett', 1993, 'Male');

INSERT INTO Movie_Actor (movie_id, actor_id, role, salary)
VALUES (1, 1, 'Lead', 15000000.00);

INSERT INTO MovieFinance (movie_id, budget, production_cost, marketing_cost,
box_office_revenue)
VALUES (1, 75000000, 62000000, 12000000, 145000000);

INSERT INTO Movie_Director (movie_id, director_id) VALUES (1, 1);
INSERT INTO Movie_Producer (movie_id, producer_id, investment) VALUES (1, 1, 75000000);
```

UPDATE — Example updates

```
UPDATE Actor SET name = 'Sofia Reyes', birth_year = 1991, gender = 'Female' WHERE id = 1;

UPDATE Movie_Actor SET role = 'Lead', salary = 16000000 WHERE movie_id = 1 AND actor_id = 1;

UPDATE MovieFinance
SET production_cost = 63000000, box_office_revenue = 150000000 WHERE movie_id = 1;

UPDATE Movie SET status = 'published',
    title = 'Midnight Siege', genre = 'Action', topic = 'Cyber Heist', release_year = 2025
WHERE id = 1;
```

DELETE — Cascade deletes

```
-- Deleting a movie cascades to all junction + finance rows
DELETE FROM Movie WHERE id = 1;

-- Remove an actor (also removes Movie_Actor rows via CASCADE)
DELETE FROM Actor WHERE id = 2;

-- Remove a specific cast assignment without deleting the person
DELETE FROM Movie_Actor WHERE movie_id = 1 AND actor_id = 3;
```

2.4 SQL Queries (Views)

Seven database views are defined in 005_queries_and_plsql.sql. Each view encapsulates a commonly needed analytical query and is consumed by the Node.js query controller.

View 1 — Full Financial Overview per Movie (with ROI)

```
CREATE OR REPLACE VIEW v_movie_financial_overview AS
SELECT
    m.id AS movie_id, m.title, m.genre, m.release_year,
    mf.budget, mf.production_cost, mf.marketing_cost,
    mf.box_office_revenue, mf.net_profit,
    CASE WHEN mf.budget > 0
         THEN ROUND((mf.net_profit / mf.budget) * 100, 2) ELSE NULL
    END AS roi_pct
FROM Movie m
JOIN MovieFinance mf ON mf.movie_id = m.id;
```

View 2 — Full Cast per Movie

```
CREATE OR REPLACE VIEW v_movie_full_cast AS
SELECT
    m.id AS movie_id, m.title AS movie_title,
    a.id AS actor_id, a.name AS actor_name, a.gender,
    ma.role, ma.salary AS actor_salary
FROM Movie m
JOIN Movie_Actor ma ON ma.movie_id = m.id
JOIN Actor       a  ON a.id = ma.actor_id;
```

View 3 — Director Filmography

```
CREATE OR REPLACE VIEW v_director_filmography AS
SELECT
    d.id AS director_id, d.name AS director_name,
    COUNT(md.movie_id) AS movie_count,
    ROUND(AVG(mf.budget), 2)     AS avg_budget,
    ROUND(AVG(mf.net_profit), 2) AS avg_net_profit
FROM Director d
LEFT JOIN Movie_Director md ON md.director_id = d.id
LEFT JOIN MovieFinance   mf ON mf.movie_id    = md.movie_id
GROUP BY d.id, d.name;
```

View 4 — Top Paid Actors

```
CREATE OR REPLACE VIEW v_top_paid_actors AS
SELECT
    a.id AS actor_id, a.name AS actor_name,
    COUNT(ma.movie_id)        AS movies_count,
    SUM(ma.salary)            AS total_earnings,
    ROUND(AVG(ma.salary), 2) AS avg_salary_per_movie
FROM Actor a
JOIN Movie_Actor ma ON ma.actor_id = a.id
WHERE ma.salary IS NOT NULL
GROUP BY a.id, a.name
ORDER BY total_earnings DESC NULLS LAST;
```

View 5 — Genre Financial Performance

```
CREATE OR REPLACE VIEW v_genre_performance AS
SELECT
    m.genre,
    COUNT(DISTINCT m.id)                  AS movie_count,
    ROUND(AVG(mf.budget), 2)              AS avg_budget,
    ROUND(SUM(mf.box_office_revenue), 2) AS total_revenue,
    ROUND(AVG(mf.net_profit), 2)          AS avg_net_profit,
    ROUND(SUM(mf.net_profit) / NULLIF(SUM(mf.budget), 0) * 100, 2) AS genre_roi_pct
FROM Movie m
JOIN MovieFinance mf ON mf.movie_id = m.id
WHERE m.genre IS NOT NULL
GROUP BY m.genre ORDER BY avg_net_profit DESC NULLS LAST;
```

View 6 — Full Crew Roster per Movie

```
CREATE OR REPLACE VIEW v_crew_roster AS
SELECT
    m.id AS movie_id, m.title AS movie_title,
    cm.id AS crew_member_id, cm.name AS crew_member_name,
    mc.job_title, mc.salary AS crew_salary
FROM Movie m
JOIN Movie_CrewMember mc ON mc.movie_id = m.id
JOIN CrewMember       cm ON cm.id = mc.crew_member_id;
```

View 7 — Profitable Movies Ranked

```
CREATE OR REPLACE VIEW v_profitable_movies AS
SELECT
    m.id AS movie_id, m.title, m.genre, m.release_year,
    mf.budget, mf.net_profit,
    RANK() OVER (ORDER BY mf.net_profit DESC) AS profit_rank
FROM Movie m
JOIN MovieFinance mf ON mf.movie_id = m.id
WHERE mf.net_profit > 0
ORDER BY mf.net_profit DESC;
```

2.5 PL/pgSQL Stored Functions (5 Blocks)

The following five stored functions demonstrate advanced PL/pgSQL: multi-table aggregation, RETURN QUERY with multiple result sets, ILIKE pattern matching, genre profitability analysis, and budget-overrun detection.

Block 1 — get_movie_total_spend(p_movie_id INT)

Calculates the complete cost breakdown for a movie by aggregating actor payroll, crew payroll, production cost, and marketing cost.

```
CREATE OR REPLACE FUNCTION get_movie_total_spend(p_movie_id INT)
RETURNS TABLE (
    movie_id INT, title VARCHAR, actor_payroll DECIMAL,
    crew_payroll DECIMAL, production_cost DECIMAL,
    marketing_cost DECIMAL, total_spend DECIMAL
) AS $$
DECLARE
    v_actor_pay DECIMAL; v_crew_pay  DECIMAL;
    v_prod_cost DECIMAL; v_mkt_cost  DECIMAL;
    v_title     VARCHAR;
BEGIN
    SELECT m.title INTO v_title FROM Movie m WHERE m.id = p_movie_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Movie with id % not found', p_movie_id;
    END IF;
    SELECT COALESCE(SUM(ma.salary), 0) INTO v_actor_pay
        FROM Movie_Actor ma WHERE ma.movie_id = p_movie_id;
    SELECT COALESCE(SUM(mc.salary), 0) INTO v_crew_pay
        FROM Movie_CrewMember mc WHERE mc.movie_id = p_movie_id;
    SELECT COALESCE(mf.production_cost, 0), COALESCE(mf.marketing_cost, 0)
        INTO v_prod_cost, v_mkt_cost
        FROM MovieFinance mf WHERE mf.movie_id = p_movie_id;
    RETURN QUERY SELECT
        p_movie_id, v_title, v_actor_pay, v_crew_pay,
        v_prod_cost, v_mkt_cost,
        v_actor_pay + v_crew_pay + v_prod_cost + v_mkt_cost;
END;
$$ LANGUAGE plpgsql;

-- Sample output (movie_id = 1, 'Midnight Siege'):
-- actor_payroll=$76M  crew=$0  prod=$62M  mkt=$12M  total=$150M
```

Block 2 — get_movie_full_roster(p_movie_id INT)

Returns every person involved in a movie: actors (with role & salary), crew (with job title & salary), directors, and producers (with investment). Uses four RETURN QUERY sub-selects within a single function body.

```
CREATE OR REPLACE FUNCTION get_movie_full_roster(p_movie_id INT)
RETURNS TABLE (
    person_type VARCHAR, person_id INT, person_name VARCHAR,
    role_detail VARCHAR, salary_or_investment DECIMAL
) AS $$
BEGIN
    RETURN QUERY
        SELECT 'Actor'::VARCHAR, a.id, a.name, ma.role, ma.salary
        FROM Movie_Actor ma JOIN Actor a ON a.id = ma.actor_id
        WHERE ma.movie_id = p_movie_id;
    RETURN QUERY
        SELECT 'Crew'::VARCHAR, cm.id, cm.name, mc.job_title, mc.salary
        FROM Movie_CrewMember mc JOIN CrewMember cm ON cm.id = mc.crew_member_id
        WHERE mc.movie_id = p_movie_id;
    RETURN QUERY
        SELECT 'Director'::VARCHAR, d.id, d.name, NULL::VARCHAR, NULL::DECIMAL
        FROM Movie_Director md JOIN Director d ON d.id = md.director_id
        WHERE md.movie_id = p_movie_id;
    RETURN QUERY
        SELECT 'Producer'::VARCHAR, p.id, p.name, NULL::VARCHAR, mp.investment
        FROM Movie_Producer mp JOIN Producer p ON p.id = mp.producer_id
        WHERE mp.movie_id = p_movie_id;
END;
$$ LANGUAGE plpgsql;

-- Sample output (movie_id = 1):
-- ('Actor',    1, 'Sofia Reyes', 'Lead',            15000000)
-- ('Crew',     3, 'Aisha Patel', 'Cinematographer',  8000000)
-- ('Director', 1, 'Elena Ward',  NULL,               NULL)
-- ('Producer', 1, 'Priya Nair',  NULL,              75000000)
```

Block 3 — get_movies_by_actor(p_actor_name VARCHAR)

Searches for movies featuring an actor using a case-insensitive partial-name match (ILIKE). Returns movie ID, title, release year, the actor's role, and salary.

```
CREATE OR REPLACE FUNCTION get_movies_by_actor(p_actor_name VARCHAR)
RETURNS TABLE (
    movie_id INT, movie_title VARCHAR,
    release_year INT, role VARCHAR, salary DECIMAL
) AS $$
BEGIN
    RETURN QUERY
        SELECT m.id, m.title, m.release_year, ma.role, ma.salary
        FROM Movie m
        JOIN Movie_Actor ma ON ma.movie_id = m.id
        JOIN Actor       a  ON a.id = ma.actor_id
        WHERE a.name ILIKE '%' || p_actor_name || '%'
        ORDER BY m.release_year DESC NULLS LAST;
END;
$$ LANGUAGE plpgsql;

-- Sample call:  SELECT * FROM get_movies_by_actor('sofia');
-- Output: (1, 'Midnight Siege', 2025, 'Lead', 15000000)
```

Block 4 — get_most_profitable_genre()

Aggregates net profit and box-office revenue by genre and returns the single most profitable genre using ORDER BY ... LIMIT 1 inside RETURN QUERY.

```
CREATE OR REPLACE FUNCTION get_most_profitable_genre()
RETURNS TABLE (
    genre VARCHAR, movie_count BIGINT,
    avg_net_profit DECIMAL, total_revenue DECIMAL
) AS $$
BEGIN
    RETURN QUERY
        SELECT
            m.genre,
            COUNT(DISTINCT m.id),
            ROUND(AVG(mf.net_profit), 2),
            ROUND(SUM(mf.box_office_revenue), 2)
        FROM Movie m
        JOIN MovieFinance mf ON mf.movie_id = m.id
        WHERE m.genre IS NOT NULL
        GROUP BY m.genre
        ORDER BY AVG(mf.net_profit) DESC NULLS LAST
        LIMIT 1;
END;
$$ LANGUAGE plpgsql;

-- Sample output: ('Action', 1, 71000000.00, 145000000.00)
```

Block 5 — flag_over_budget_movies()

Identifies movies where actual production cost exceeded the approved budget. Computes the absolute overrun and the percentage overrun using a CASE expression.

```
CREATE OR REPLACE FUNCTION flag_over_budget_movies()
RETURNS TABLE (
    movie_id INT, title VARCHAR,
    budget DECIMAL, production_cost DECIMAL,
    overrun DECIMAL, overrun_pct DECIMAL
) AS $$
BEGIN
    RETURN QUERY
        SELECT
            m.id, m.title,
            mf.budget, mf.production_cost,
            mf.production_cost - mf.budget,
            CASE WHEN mf.budget > 0
                 THEN ROUND(((mf.production_cost - mf.budget) / mf.budget) * 100, 2)
                 ELSE NULL
            END
        FROM Movie m
        JOIN MovieFinance mf ON mf.movie_id = m.id
        WHERE mf.production_cost > mf.budget
        ORDER BY (mf.production_cost - mf.budget) DESC;
END;
$$ LANGUAGE plpgsql;

-- All movies in current dataset are within budget -> returns empty set
```

3. Software

The application is an Electron desktop app written in ES Module JavaScript. The main process (src/app.js) handles all IPC channels and database calls; the renderer process (src/UI/renderer.js) handles the UI; a context bridge in src/UI/preload.cjs exposes a safe API to the renderer without enabling node integration.

3.1 Database Connection

src/services/neonClient.js

```
import { neon } from '@neondatabase/serverless';
import dotenv from 'dotenv';

dotenv.config();

const sql = neon(process.env.DATABASE_URL);
export default sql;
```

.env (credentials redacted for report)

```
DATABASE_URL='postgresql://<user>:<password>@<host>/neondb?sslmode=require&channel_binding=r
equire'
ADMIN_USERNAME='admin'
ADMIN_PASSWORD='admin123'
```

The neon() factory returns a tagged-template SQL executor that sends parameterised queries to Neon's serverless HTTP endpoint. This eliminates the need for a persistent TCP connection. All controllers import the same singleton sql object.

3.2 Architecture Overview

Layer	File(s)	Responsibility
Electron Main	src/app.js	Window creation, IPC handlers, app lifecycle
Controllers	src/controllers/*.js	Thin wrappers — call PL/pgSQL functions via sql``
DB Client	src/services/neonClient.js	Neon serverless connection singleton
UI – HTML	src/UI/index.html	Single-page application shell
UI – JS	src/UI/renderer.js	DOM manipulation, IPC calls via window.electronAPI
UI – CSS	src/UI/styles.css	Dark terminal-aesthetic theme
Preload	src/UI/preload.cjs	Context bridge — exposes electronAPI to renderer
Migrations	migrations/*.sql	Schema creation, functions, views, seed data
Runner	migrations/migrate.js	Reads & executes all *.sql files in order

3.3 Application Screenshots

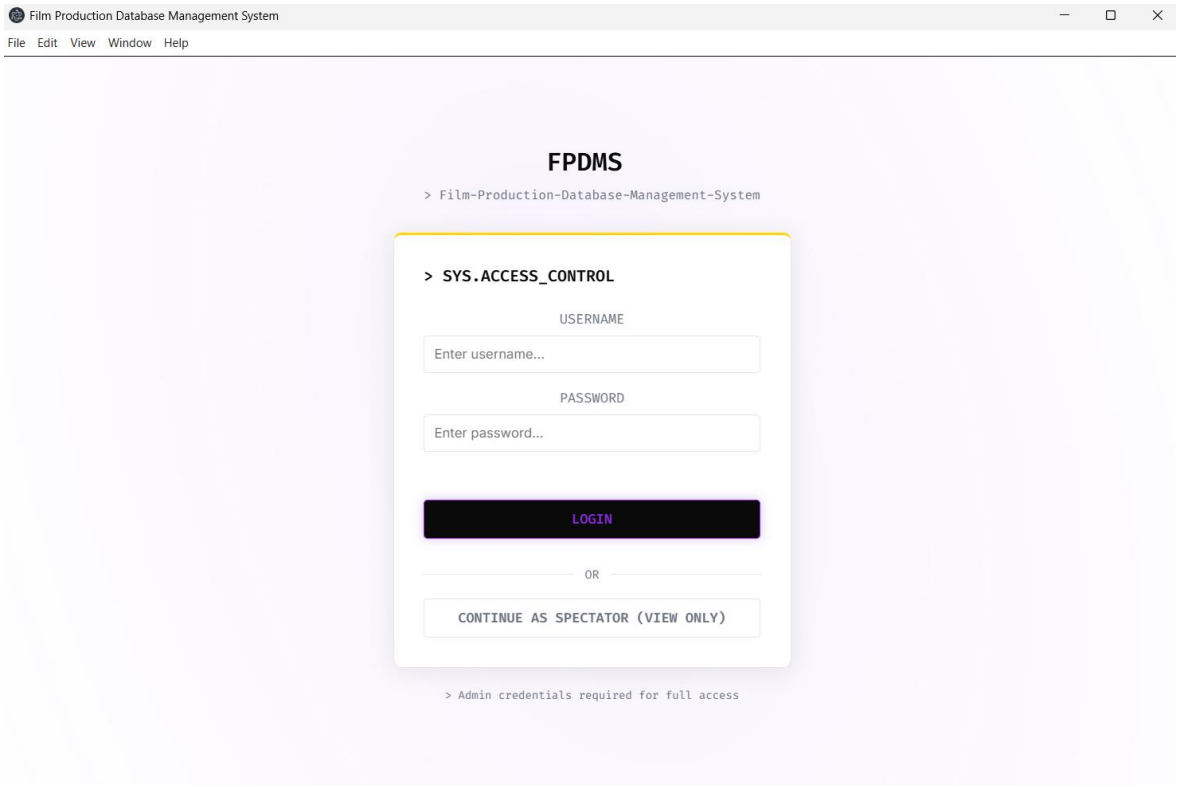


Figure 2 – Login Screen (SYS.ACCESS_CONTROL)

The entry point of the application. The user can log in as Admin (full CRUD access) or continue as a Spectator (read-only). Admin credentials are validated against environment variables, keeping them out of the database.

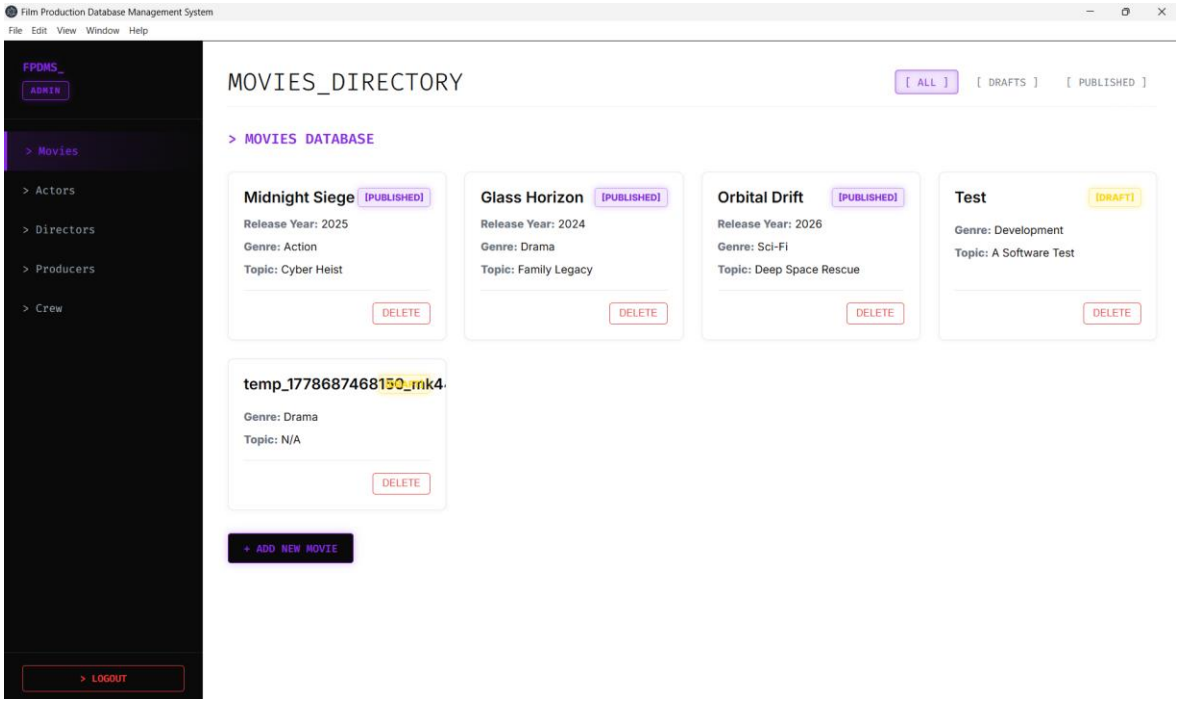


Figure 3 – Movies Directory (MOVIES_DIRECTORY)

Shows all movies as cards. Each card displays the title, status badge (PUBLISHED / DRAFT), genre, release year, and topic. Filter buttons at the top allow filtering by status. Admin users see a DELETE button and an ADD NEW MOVIE button.

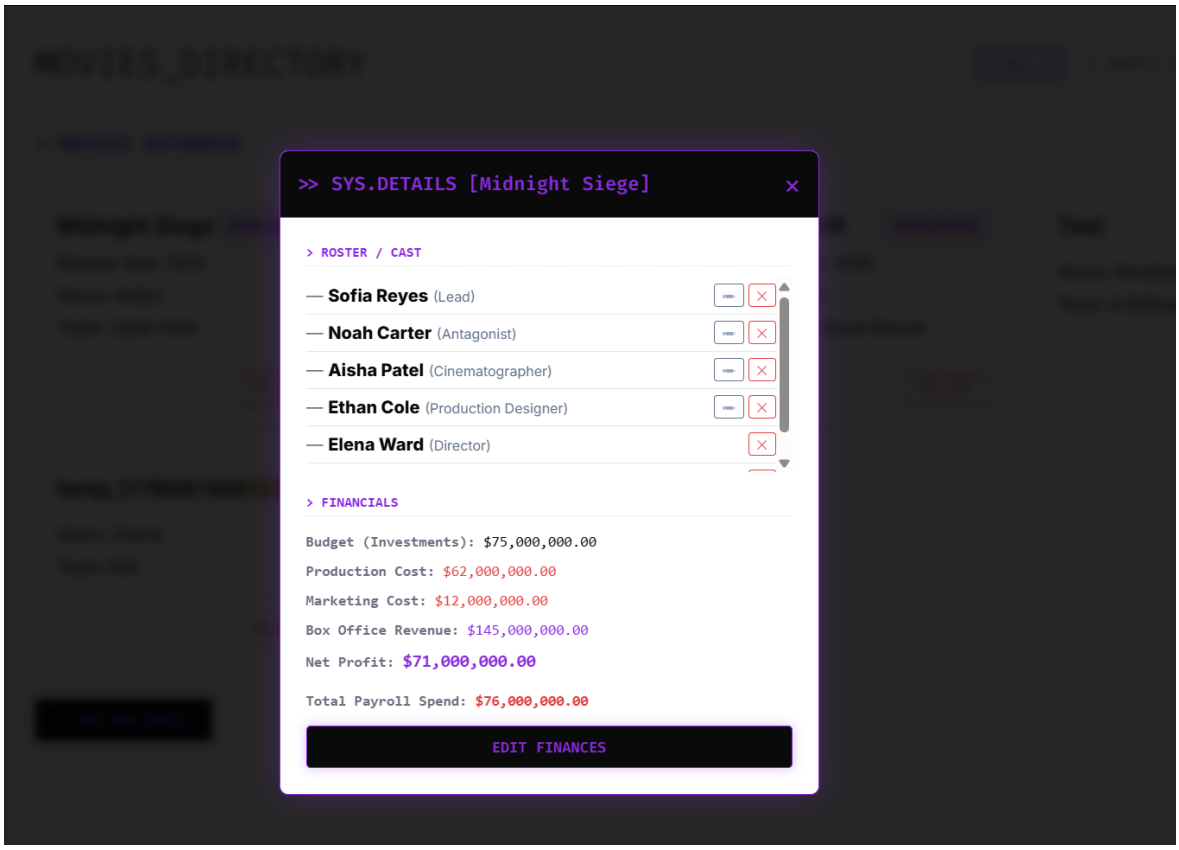


Figure 4 – Movie Detail Modal (SYS.DETAILS)

Clicking a movie opens this modal. The ROSTER/CAST section lists every person involved fetched via get_movie_full_roster(). The FINANCIALS section shows Budget, Production Cost, Marketing Cost, Box Office Revenue, Net Profit, and Total Payroll computed by get_movie_total_spend().

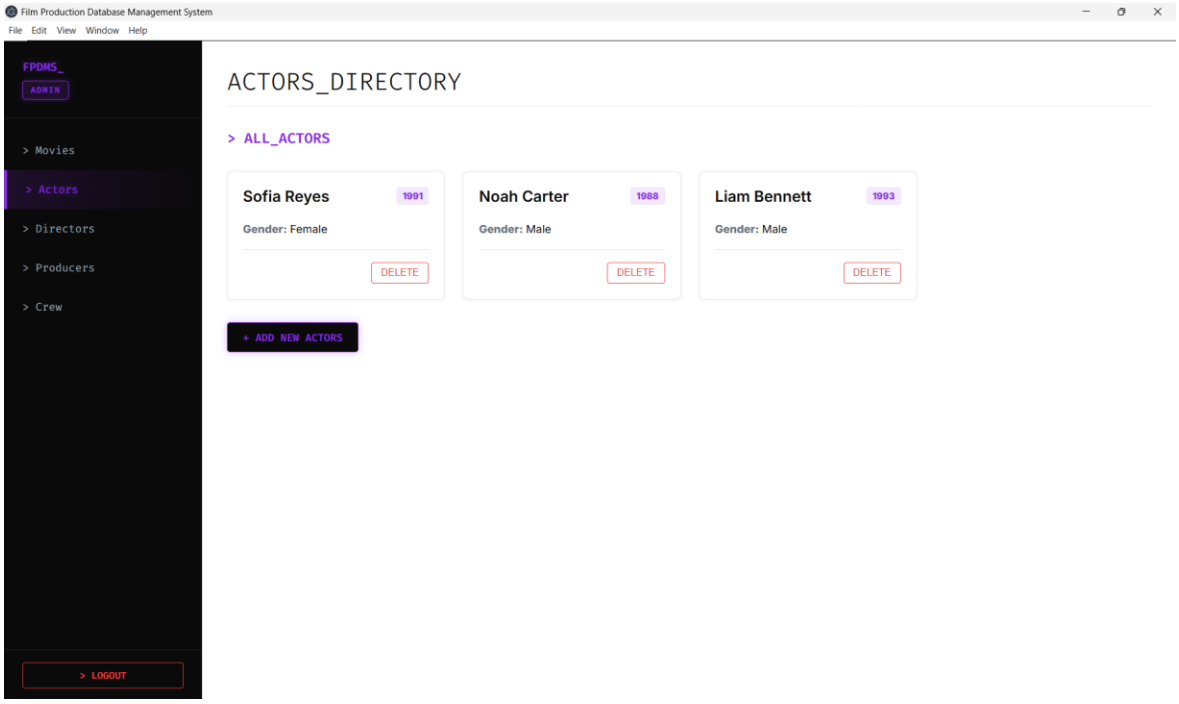


Figure 5 – Actors Directory (ACTORS_DIRECTORY)

Displays all registered actors with birth year and gender. Admin users can delete actors (cascades to all Movie_Actor rows) or add new actors via the ADD NEW ACTORS button, which calls the add_actor() PL/pgSQL function.

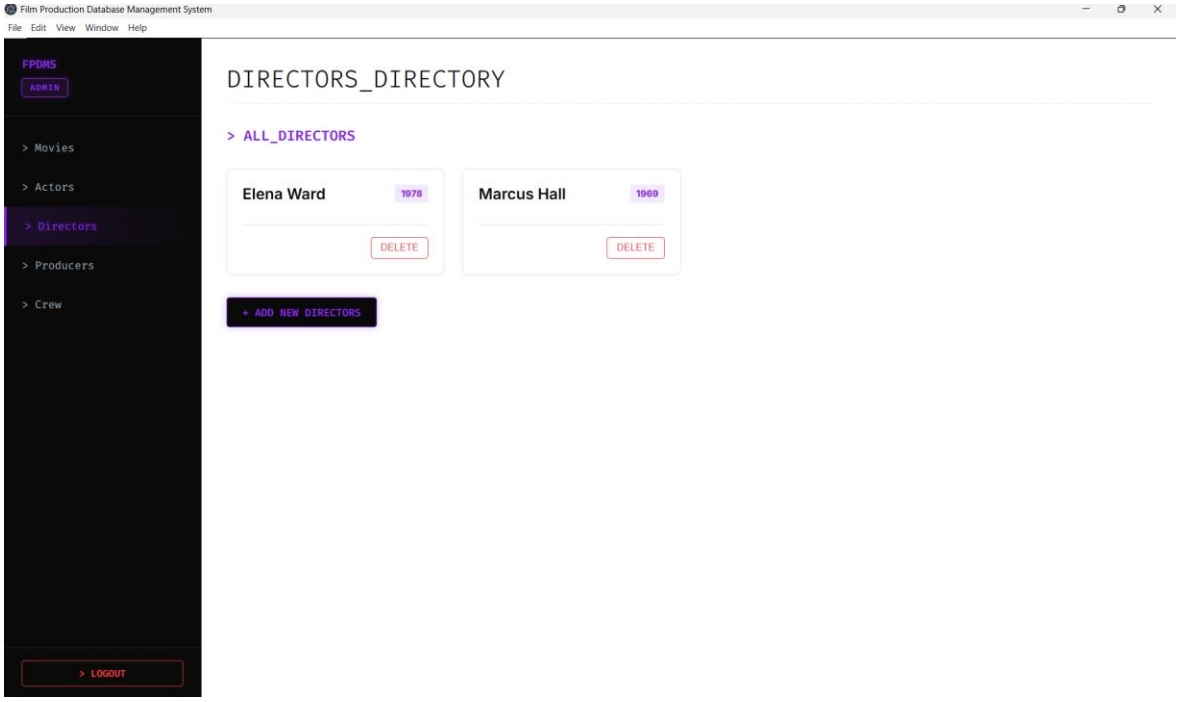


Figure 6 – Directors Directory (DIRECTORS_DIRECTORY)

Lists all directors. ROI statistics are available via get_director_roi(). Add and delete operations are available to admin users.

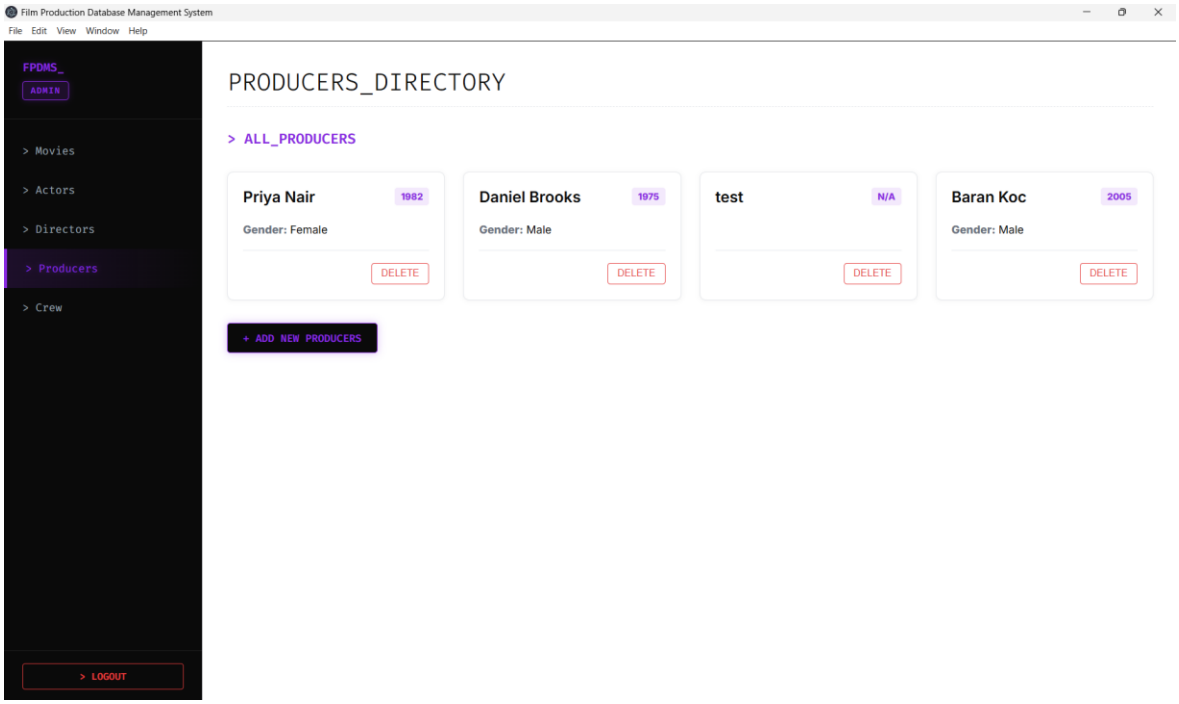


Figure 7 – Producers Directory (PRODUCERS_DIRECTORY)

Shows all producers with birth year and gender. Producers are linked to movies through Movie_Producer with an investment amount that feeds the MovieFinance.budget column.

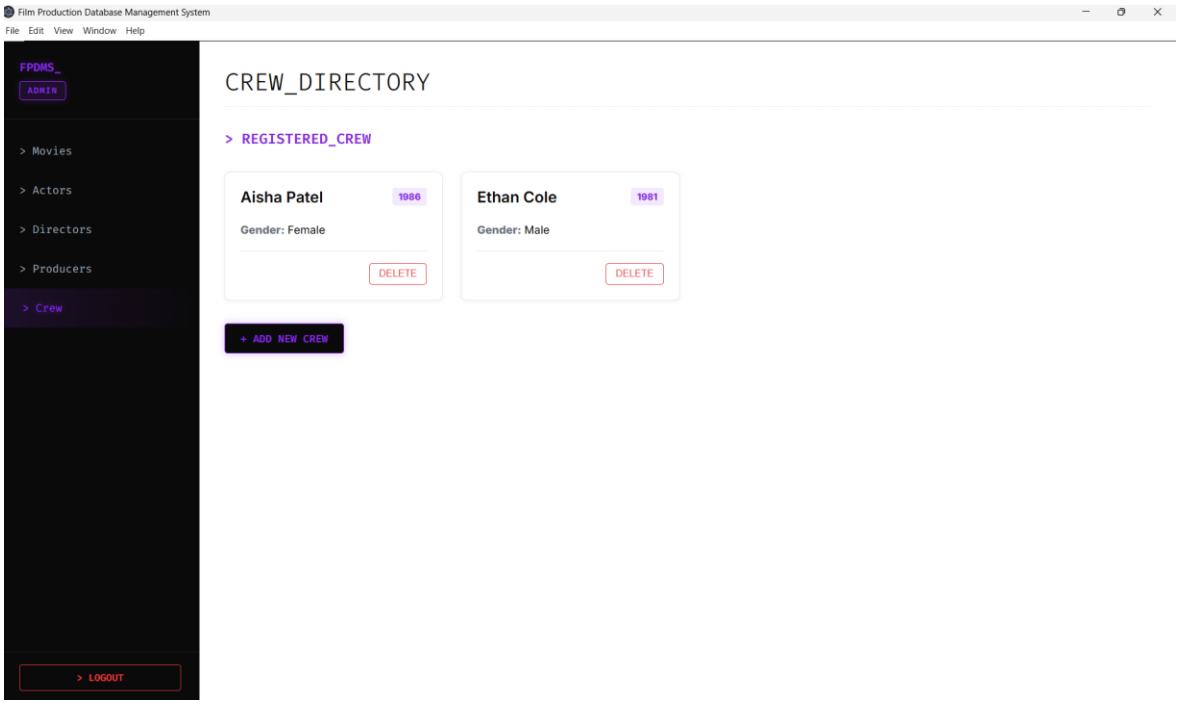


Figure 8 – Crew Directory (CREW_DIRECTORY)

Lists registered crew members. Each crew member can be assigned a job title and salary per-movie through the Movie_CrewMember junction table.

4. GitHub Repository

The full source code for this project is publicly available on GitHub:

```
https://github.com/22314825/Film-Production-Database-Management-System
```

Repository Structure

Path	Description
src/app.js	Electron main process — window + IPC handlers
src/controllers/	Controller modules for each entity (movie, actor, director, producer, crew, finance, query)
src/services/neonClient.js	Neon serverless client singleton
src/UI/	Frontend HTML, CSS, JavaScript (renderer + preload)
src/test/	Automated test suite (runAllTests.js + per-entity tests)
migrations/	SQL migration files (001–006) + migrate.js / reset.js / setup.js runners
.env	Environment variables (not committed — listed in .gitignore)
package.json	NPM manifest — scripts: start, migrate, reset, setup, test

Documentation Details

- README.md — Setup instructions: cloning the repo, installing dependencies (npm install), configuring .env, running migrations (npm run migrate), and starting the app (npm start).
- migrations/ — Each SQL file is self-contained and idempotent where possible (CREATE OR REPLACE for functions and views). Files are numbered and run in order.
- src/test/runAllTests.js — A Node.js test runner that exercises every controller method, verifying insert, read, update, and delete operations against the live Neon database.
- The repository is set to public visibility to allow assessment access.
- Commit history reflects contributions from all four team members across the development lifecycle: schema design, backend functions, UI implementation, testing, and documentation.

How to Run

```
# 1. Clone the repository
git clone https://github.com/22314825/Film-Production-Database-Management-System
cd Film-Production-Database-Management-System

# 2. Install dependencies
npm install

# 3. Create .env file with your Neon DATABASE_URL

# 4. Run database migrations (creates schema + seeds data)
npm run migrate

# 5. Launch the Electron application
npm start

# Optional: run all tests
npm test
```